

Projektantrag: SaldoFlow – Modulares Haushaltsbuch

Projektbezeichnung

SaldoFlow – Modulares Haushaltsbuch

(Entwicklung einer schichtenbasierten, objektorientierten Python-/Flask-Webanwendung zur privaten Finanzverwaltung)

Auftraggeber

- Dozenten: Teichert, Wilfried Klassen: US IT 2026 Winter FIAE B.1

Startdatum

- [07.09.2026]

Ziele

Hauptziel:

Entwicklung einer lauffähigen, wartbaren Webanwendung zur Erfassung und Auswertung privater Finanzen. Der Schwerpunkt liegt auf der Einhaltung professioneller Software-Engineering-Standards (Clean Code, OOP, src/-Layout und Schichtentrennung) sowie einer barrierefreien Ausführbarkeit ohne komplexe Entwicklungsumgebung.

Teilziele:

1. Domänenmodellierung (Domain Layer / OOP):

- Saubere Klassenarchitektur in Python ohne Framework-Abhängigkeiten (Transaction, Subklassen Income und Expense, Aggregat BudgetBook).
- Kapselung von Berechnungen (Einnahmen-, Ausgaben- und Saldo-Ermittlung).

2. Datenpersistenz & Entkopplung (Persistence Layer):

- Lokale SQLite-Datenbank (sqlite3).
- Umsetzung des *Repository-Patterns* (TransactionRepository), um Datenbankschicht und Business-Logik strikt zu trennen.

3. Web-Präsentation & Routing (Presentation Layer):

- Schlankes Flask-Backend unter Nutzung des *Application-Factory-Musters* (create_app).
- Responsives Frontend mittels semantischem HTML5 und Pico.css (ohne JavaScript-Frameworks).

4. Usability & Zero-Configuration Execution:

- Bereitstellung von 1-Klick-Start-Skripten (start.bat / start.sh), die Abhängigkeiten prüfen und die Web-App automatisch im Standardbrowser öffnen.

5. Qualitätssicherung & Dokumentation:

- Erstellung von automatisierten Unittests (tests/) für die Finanzmathematik und Repository-Methoden.
- Versionskontrolle über Git mit Feature-Branching und nachvollziehbarem Commit-Verlauf aller vier Teammitglieder.

Nicht-Ziele (Out of Scope)

- **Keine Online-Bankenschnittstellen:** Keine Anbindung an PSD2-, FinTS- oder Open-Banking-APIs.
- **Kein Multi-User- / Rollensystem:** Die Anwendung ist als Single-User-Lösung für den lokalen Betrieb konzipiert.
- **Keine OCR-/Belegerkennung:** Keine automatische Belegerfassung über Bilderkennung oder

Machine Learning.

- **Kein komplexes JavaScript-Frontend:** Kein Einsatz von Frameworks wie React oder Vue; Fokus liegt rein auf Python/Flask mit server-seitigem Rendering (Jinja2).

Begründung

Klassische Finanzverwaltungstools sind im Alltag oft überladen oder intransparent. Die Entwicklung von SaldoFlow dient als praxisnahes Lehrprojekt, um die Phasen des modernen Software-Engineerings im Team zu demonstrieren: Von der Anforderungsanalyse über die Architekturentscheidung (3-Schichten-Modell) bis hin zur industriekonformen Paketierung in Python. Durch die Entkopplung von Kernlogik, Persistenz und Web-UI wird ein hoher Standard an Wartbarkeit und Testbarkeit sichergestellt.

Gewünschtes Projektende

- [TT.MM.JJJJ]

Kostenschätzung

- **Finanzielle Kosten:** 0,00 € (Ausschließliche Nutzung freier Open-Source-Technologien: Python 3, Flask, SQLite, Pico.css, Git).
- **Personeller Gesamtaufwand:**
 - 4 Personen × ca. [40] Arbeitsstunden = ca. [160] Personenstunden Projektbudget.

Projektleiter & Rollenverteilung

- **Projektleitung & Systemarchitektur:** [Name Mitglied 1]

(Aufgaben: Projektplanung, Git-Workflow, Application Factory & Flask-Setup, Start-Skripte)

- **Domänenlogik & OOP (Model Layer):** [Name Mitglied 2]

(Aufgaben: Klassen Transaction, Income, Expense, BudgetBook, Validierungsregeln)

- **Datenpersistenz & Testing (Persistence Layer):** [Name Mitglied 3]

(Aufgaben: SQLite-Datenbankschema, TransactionRepository, Unittests im Ordner tests/)

- **Frontend & UX (Presentation Layer):** [Name Mitglied 4]

(Aufgaben: Jinja2-Templates, Pico.css-Integration, Formular- und Dashboard-Design)

Kontrollinstanz

- **Fachliche Kontrolle:**
 - Dozenten: Teichert, Wilfried
 - Klasse: US IT 2026 Winter FIAE B.1

(Abnahme der Meilensteine: Anforderungsspezifikation, Architektur-Review, Zwischenpräsentation, Endabnahme)

- **Interne Kontrolle:** Wöchentliche Team-Standup-Meetings und Code-Reviews vor Merges in den Hauptzweig (main).

Kriterien für das Projektende

1. **Lauffähigkeit:** Die App lässt sich auf einem Zielrechner (Windows oder macOS/Linux) mittels start.bat bzw. start.sh per Doppelklick ohne manuelle Konfiguration starten und öffnet das Browser-Dashboard.
2. **Funktionsumfang:** Sämtliche Transaktionen (Einnahmen und Ausgaben) können über das Frontend erfasst, validiert, dauerhaft in SQLite gespeichert, angezeigt und gelöscht werden; Saldo und Summen berechnen sich korrekt.

3. **Architekturtreue:** Der Code liegt im industriekonformen src/-Layout vor. Die Domänenlogik ist strikt von Flask und SQLite getrennt (Repository-Pattern).
4. **Qualität:** Alle Unittests für die OOP-Kernfunktionen laufen ohne Fehler durch (pytest / unittest).
5. **Dokumentation:** Vollständige Projektdokumentation inklusive Klassendiagramm, Schichtenarchitektur-Übersicht und Benutzeranleitung in der README.md.